

JavaScript: Prototypes vs Classes

1. Prototype-based approach

- Every function in JS has a hidden property called prototype.
- When you create objects with a constructor function, methods are defined on that function's prototype so all instances share them.

Example:

```
function Person(name) {  
  this.name = name; // instance property  
}
```

```
Person.prototype.greet = function() {  
  console.log("Hello, I'm " + this.name);  
};
```

```
const p1 = new Person("Alice");  
p1.greet(); // Hello, I'm Alice
```

2. Class-based approach (ES6+)

- Classes are syntactic sugar for the prototype mechanism.
- They make code more readable and structured like other OOP languages.

Example:

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  greet() {  
    console.log("Hello, I'm " + this.name);  
  }  
}
```

```
const p1 = new Person("Alice");  
p1.greet(); // Hello, I'm Alice
```

3. Key Differences

Prototype vs Class

- Syntax: function + prototype vs cleaner class keyword
- Readability: verbose vs intuitive
- Inheritance: Object.create/manual vs extends keyword
- Concept: core mechanism vs syntactic sugar

4. Example of Inheritance

Prototype way:

```
function Animal(name) {  
  this.name = name;  
}  
Animal.prototype.speak = function() {  
  console.log(this.name + " makes a noise.");  
};  
  
function Dog(name) {  
  Animal.call(this, name);  
}  
Dog.prototype = Object.create(Animal.prototype);  
Dog.prototype.constructor = Dog;  
Dog.prototype.speak = function() {  
  console.log(this.name + " barks.");  
};
```

Class way:

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  speak() {  
    console.log(this.name + " makes a noise.");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log(this.name + " barks.");  
  }  
}
```

Conclusion

- Prototypes are the underlying mechanism of object behavior in JavaScript.
- Classes provide a cleaner and more structured syntax, but internally rely on prototypes.